

Laboratorio II

Corso A

Esercitazione 2

- Programmazione C avanzata
 - Puntatori
 - Array multidimensionali
 - Allocazione dinamica di memoria
 - Chiamata di funzioni

Esercizi

Data una matrice quadrata A di dimensione MxM, calcolare la somma degli elementi di una sottomatrice quadrata B **di testa** di dimensione NxN usando una funzione `sum_submatrix` con prototipo :

```
int sum_submatrix(int **matrix, int n)
```

con $M \geq N \geq 0$ (con sottomatrice di testa si intende la sottomatrice che inizia dall'angolo superiore sinistro della matrice A). Si allochi la matrice A dinamicamente, per righe.

Inserire dallo stdin prima la dimensione della matrice A, seguita dalla dimensione della matrice B, e poi gli elementi della matrice A, inseriti per righe, uno per ogni riga dello stdin. Stampare in output, in una funzione, il valore della somma degli elementi della matrice di testa B.

Esempio di input:

4 (M)

2 (N)

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

Esempio di output:

14

Esercizi: soluzione 1

```
#include <stdio.h>
#include <stdlib.h>

int sum_submatrix(int **matrix, int n)
{
    int sum = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            sum += matrix[i][j];
        }
    }
    return sum;
}

void print_matrix(int **matrix, int m)
{
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            printf("%3d ",
matrix[i][j]);
        }
        printf("\n");
    }
}
```

```
int main() {
    int n, m;
    // Leggi un intero n dall'input standard (stdin), dimensione della
matrice
    scanf("%d\n", &m);

    // Dimensione matrice
    if (m == 0){
        return 0;
    }
    //Dimensione sottomatrice
    scanf("%d\n", &n);
    if (n == 0){
        return 0;
    }

    if (n>m){
        return 0;
    }

    // Dichiarazione e allocazione dinamica della matrice M di dimensioni
m x m
    int** A = (int **)malloc(m * sizeof(int*));
    for (int i = 0; i < m; i++){
        A[i] = (int *) malloc(m * sizeof(int));
    }

    // Leggi i valori della matrice A
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d\n", &A[i][j]);
        }
    }
    printf("%d\n", sum_submatrix(A, n));
    return 0;
}
```

Esercizi

Data una matrice quadrata A di dimensione $M \times M$, inserire dallo stdin prima la dimensione della matrice A , e poi gli elementi della matrice A , inseriti per righe, uno per ogni riga dello stdin.

Usando una funzione, cambiare il segno dei valori sulla diagonale principale della matrice A e stampare infine il contenuto della matrice, una riga per ogni riga dello stdout, come nel seguente esempio.

Esempio di Input:

```
3
1
2
3
4
5
6
7
8
9
```

Esempio di output:

```
-1 2 3
4 -5 6
7 8 -9
```

Esercizi: soluzione 2

```
#include <stdio.h>
#include <stdlib.h>

void cambia_segno_matrice(int **matrix,
int N) {
    for (int i = 0; i < N; i++) {
        matrix[i][i] *= -1; // Cambia
il segno del valore sulla diagonale
    }
}

void print_matrix(int **matrix, int N)
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ",
matrix[i][j]);
        }
        printf("\n");
    }
}
```

```
int main() {

    int n;
    // Leggi un intero n dall'input standard (stdin), dimensione
della matrice
    scanf("%d", &n);

    if (n == 0){
        return 0;
    }

    // Dichiarazione e allocazione dinamica della matrice M di
dimensioni m x m
    int** M = malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++){
        M[i] = malloc(n * sizeof(int));
    }
    // Leggi i valori della matrice M
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d\n", &M[i][j]);
        }
    }
    cambia_segno_matrice(M, n);
    print_matrix(M, n);
    for (int i = 0; i < N; i++) {
        {
            free(matrix[i]);
        }
    }

    return 0;
}
```

Esercizi

Dato un array di numeri interi, scrivere una funzione *somma_con_trasformazione* che calcoli la somma degli elementi di questo array. Questa funzione prende un puntatore a una funzione come parametro, ed ha il seguente prototipo, data la seguente definizione di tipo custom:

```
typedef int (*trasforma_fn)(int);
```

e il seguente prototipo per la funzione:

```
int somma_con_trasformazione (int* a, int dim, transforma_fn trasforma)
```

La funzione passata come parametro determina come ogni elemento dell'array dovrebbe essere trasformato prima di essere sommato. Le due possibili trasformazioni sono:

- *valore_assoluto*: calcola il valore assoluto dell'intero passato come parametro
- *quadrato*: calcola il quadrato dell'intero dell'intero passato come parametro

Il primo valore fornito da tastiera rappresenta la dimensione dell'array. Utilizzare l'allocazione dinamica della memoria.

Nello stampare il risultato, utilizzare prima la funzione con il valore assoluto, seguita da uno spazio e poi dalla funzione che calcola il quadrato.

Esempio di input:

```
5
1
-1
1
-1
0
```

Esempio di output:

```
4 4
```

Esercizi: soluzione 3

```
#include <stdio.h>
#include <stdlib.h>

typedef int (*trasforma_fn)(int);

int valore_assoluto(int a) {
    if (a < 0) return -a;
    else return a;
}

int quadrato(int a) {
    return a * a;
}

int somma_con_trasformazione (int* arr, int dim,
    trasforma_fn trasforma) {
    int somma = 0;
    for (int i = 0; i < dim; i++) {
        somma += trasforma(arr[i]);
    }
    return somma;
}
```

```
int main() {
    int dim;

    scanf("%d", &dim);

    if (dim == 0){
        return 0;
    }

    int* A = (int*)malloc(dim * sizeof(int));

    for (int i = 0; i < dim; i++) {
        scanf("%d\n", &A[i]);
    }

    int total = somma_con_trasformazione(A, dim,
    valore_assoluto);
    printf("%d ", total);

    total = somma_con_trasformazione (A, dim,
    quadrato);
    printf("%d\n", total);

    return 0;
}
```


Esercizi

Scrivere un programma che lavori su una lista concatenata, costituita da elementi interi, il cui inserimento termina appena viene digitato un numero negativo.

Le operazioni da implementare devono essere contenute in apposite funzioni e essere le seguenti, dato un puntatore alla testa della lista denominato L:

-*print*(L): stampa tutti gli elementi della lista L, uno per riga

-*insert*(x, L): inserire l'elemento in coda alla lista L (nota: NON RICORSIVA)

-*insertRic*(int x, L): come la precedente MA RICORSIVA

-*delete*(L): elimina dalla lista L l'ultimo elemento, NN RICORSIVA

Nella funzione `main`, testare le funzioni implementate, inserendo un elemento della lista per riga. Usare la funzione ricorsiva se l'elemento e' pari, non ricorsiva se e' dispari. Stampare in output gli elementi della lista, uno per riga. Dopo aver inserito l'elemento negativo, svuotare la lista e stamparla nuovamente.

Esempio di Input:

4
30
22
124
-2

Esempio di Output:

4
30
22
124
-2

Esercizi: soluzione 4

```
#include <stdio.h>
#include <stdlib.h>

typedef struct n {
    int val;
    struct n * next;
} Nodo;

typedef Nodo* Lista;

void print(Lista );

void insert(int, Lista*);

void insertRic(int, Lista*);

void delete (int, Lista*);
```

```
int main(){

    Lista l=NULL;
    int x, num_el=0;
    do{
        scanf("%d",&x);
        if(x%2 == 0)
            insertRic(x,&l);
        else
            insert(x,&l);

    }while(x>=0);
    print(l);
    scanf("%d",&x);
    for(int i=0;i
    delete(x,&l);
    print(l);
}

/* Inserisce, ricorsivamente, un elemento in fondo lista */
void insertRic(int x, Lista* l){

    if ((*l)==NULL){
        Nodo* nuovo=(Nodo*)malloc(sizeof(Nodo));

        if(nuovo==NULL)
            exit(1);

        nuovo->val=x;
        nuovo->next=*l;

        *l=nuovo;
    }
    else
        insertRic(x,&((*l)->next));
}
```

Esercizi: soluzione 4

```
/* stampa tutti gli elementi di una lista, partendo dalla testa,
ricorsivamente */
void print(Lista l){
    if(l==NULL)
        return;
    printf("%d\n",l->val);
    print(l->next);
}

void delete (int x, Lista* L)
{
    if(*L==NULL)
        return;
    Nodo *attuale = *L;
    Nodo *prec = NULL;

    while (attuale->next!=NULL){
        prec=attuale;
        attuale = attuale ->next;
    }

    free(attuale);
    if(prec==*L)
        *L=NULL;
    else
        prec->next=NULL;
}
```

```
/* Inserisce un elemento in fondo alla lista ( non ricorsiva ) */
void insert(int x, Lista* l){

    Nodo* prec, *succ, *nuovo;

    nuovo=(Nodo*)malloc(sizeof(Nodo));

    if (nuovo==NULL)
        exit(1);

    nuovo->val=x;
    nuovo->next=NULL;

    prec=NULL;
    succ=*l;

    while (succ!=NULL) {
        prec=succ;
        succ=succ->next;
    }

    nuovo->next=succ;
    if(prec!=NULL)
        prec->next=nuovo;
    else
        *l=nuovo;
}
```

Esercizi

Utilizzando l'implementazione vista a lezione delle liste concatenate, si deve implementare una struttura dati rappresentante una *pila* (struttura dati LIFO) e una serie di funzioni che permettono le seguenti operazioni:

- push* - inserisce un elemento nella *pila*
- pop* - rimuove il primo elemento dalla *pila* e lo ritorna. Deve tornare il valore 0 se la *pila* è vuota.
- peek* - restituisce il valore del primo elemento della *pila* senza rimuoverlo. Deve tornare il valore 0 se la *pila* è vuota.
- length* - calcola la lunghezza della *pila* **in modo ricorsivo**
- print* - stampa gli elementi della *pila* , iniziando con l'elemento in testa (vedi esempio), **in modo ricorsivo**

Scrivere una funzione **Main** che legge da standard input una serie di numeri interi e, utilizzando le funzioni definite sopra, esegue le seguenti operazioni:

- Se il numero è diverso da zero ed è un multiplo di 3, viene inserito nella *pila* il numero diviso 3
- Se il numero è diverso da 0 e non è un multiplo di 3, viene inserito in *pila*
- Se il numero è 0, viene eliminato un elemento dalla *pila* , solo se l'elemento da eliminare è dispari o la *pila* attualmente contiene più di 3 elementi.
- Il programma si ferma quando si leggono tre valori 0 di seguito

Alla fine della funzione **Main**, deve essere stampato in stdout il contenuto della *pila* e la sua lunghezza, con tutti gli elementi della *pila* su un'unica riga, separati da uno spazio, e la lunghezza della *pila* come ultimo valore.

Esempio di input:

```
2
56
-3
5
897
432
0
19
-54
0
45
0
0
0
```

Esempio di output:

```
5 -1 56 2 4
```

Esercizi: soluzione 5

```
#include <stdio.h>
#include <stdlib.h>

typedef struct ll_node_struct{
    int info;
    struct ll_node_struct* next;
} Node;

typedef Node* Pila;

void print(Pila head)
{
    if (head==NULL)
        return;
    printf("%d ",head->info);
    print(head->next);
}
```

```
// Funzione push: prende come parametro la testa della pila
// e un intero e aggiunge in cima alla pila un nuovo nodo
// Ritorna il puntatore alla nuova pila
void push(Pila* head, int v)
{
    //creiamo un nuovo nodo
    Node* node=malloc(sizeof(Node));
    node->info=v;
    node->next=*head;
    *head=node;
}

// Funzione pop: prende come parametro la testa della pila
// e rimuove il primo elemento dalla pila. Ritorna il valore
// contenuto nel nodo estratto, o 0 se la pila è vuota.
int pop(Pila *head)
{
    if(*head==NULL)
        return 0;
    int res = (*head)->info;
    Pila old_head = *head;
    *head = (*head)->next;
    free(old_head);
    return res;
}
```

Esercizi: soluzione 5

```
// Funzione peek: prende come parametro la testa della
pila.
// Ritorna il valore contenuto nel primo nodo, o 0 se la
pila è vuota.
int peek(Pila head)
{
    if(head==NULL)
        return 0;

    return head->info;
}

// Funzione length: ritorna la lunghezza della pila
int length(Pila head)
{
    if (head==NULL)
        return 0;

    return 1 + length(head->next);
}
```

```
int main()
{

    Pila head=NULL;
    int valore, zero_trovati=0;

    do
    {
        scanf("%d",&valore);
        if((valore%3)==0 && valore!=0)
        {
            zero_trovati=0;
            push(&head,valore/3);
        }
        if(valore%3!=0 && valore!=0)
        {
            zero_trovati=0;
            push(&head, valore);
        }

        if(valore==0)
        {
            zero_trovati++;
            if(peek(head)%2==1 || length(head)>3)
                pop(&head);
        }
    }
    while(zero_trovati<3);
    print(head);
    printf("%d",length(head));
    return 0;
}
```